

Primer Parcial Práctico

Cinemática Directa e Inversa del Robot

KUKA KR 6 R900 sixx con ROS 2 Jazzy

Grupo 3

Fernando Aramayo

Cristina Montaña

IMT-342 Robótica – Universidad Católica Boliviana “San Pablo”

Docente: Carlos Daniel Aguilar Mancachi

Resumen—Se presenta el modelado cinemático del robot KUKA KR 6 R900 sixx, un manipulador serial de seis articulaciones revolutas. A partir de las dimensiones del modelo y de los marcos de referencia se establecen los parámetros Denavit–Hartenberg. Se desarrolla la cinemática directa para calcular posición y orientación, y se obtiene el Jacobiano posicional mediante la formulación geométrica. La cinemática inversa de posición se implementa con un método iterativo basado en la pseudoinversa, límites articulares y una tolerancia de un milímetro. Los nodos de ROS 2 reciben objetivos cartesianos y publican estados articulares para visualizar el robot en RViz2. Las tres pruebas documentadas de cinemática inversa reportan errores inferiores a la tolerancia.

Index Terms—Denavit–Hartenberg, cinemática directa, cinemática inversa, Jacobiano, ROS 2, RViz2, manipulador serial.

I. DESCRIPCIÓN DEL ROBOT ASIGNADO

El robot dispone de seis ejes, una carga máxima de 6 kg y un alcance nominal de 901 mm según el fabricante [1]. El modelo ROS utilizado pertenece al paquete `kuka_agilus_support` [2].

I-A. Datos generales

Cuadro I
DATOS PRINCIPALES DEL ROBOT

Dato	Valor
Fabricante	KUKA
Modelo	KR 6 R900 sixx
Grados de libertad	6
Tipo de articulaciones	Revolutas
Marco base	<code>base_link</code>
Efector final	<code>tool0</code>
Unidades empleadas	SI: metros y radianes

I-B. Cadena cinemática

En la siguiente imagen se pueden observar las 6 articulaciones revolutas que contiene el robot.

II. ASIGNACIÓN DE MARCOS DE REFERENCIA

La figura 2 muestra la asignación de marcos de referencia para el robot.

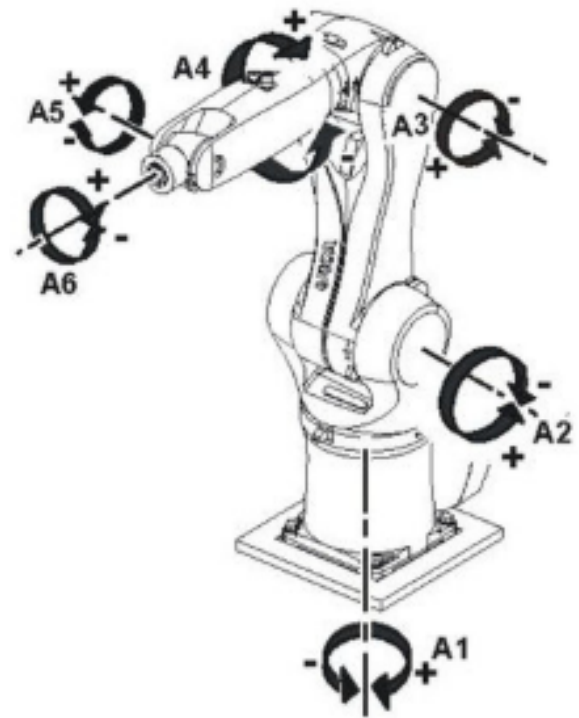


Figura 1. Robot KUKA KR6 R900 y articulaciones consideradas.

En DH estándar, x_{i+1} sigue la normal común entre z_i y z_{i+1} , orientada según el desplazamiento firmado de la tabla. Para ejes que se intersectan se elige perpendicular a ambos; si son coincidentes, su elección se fija mediante el marco de herramienta. Se completa cada triedro con $y_i = z_i \times x_i$ y los ángulos positivos siguen la regla de la mano derecha. La figura 2 muestra la asignación de marcos de referencia para el robot, contemplando las 6 articulaciones y adicionalmente 2 marcos de referencia adicionales, uno para la base y el otro para la herramienta.

II-A. Convención utilizada

En este trabajo se utiliza la convención Denavit–Hartenberg estándar:

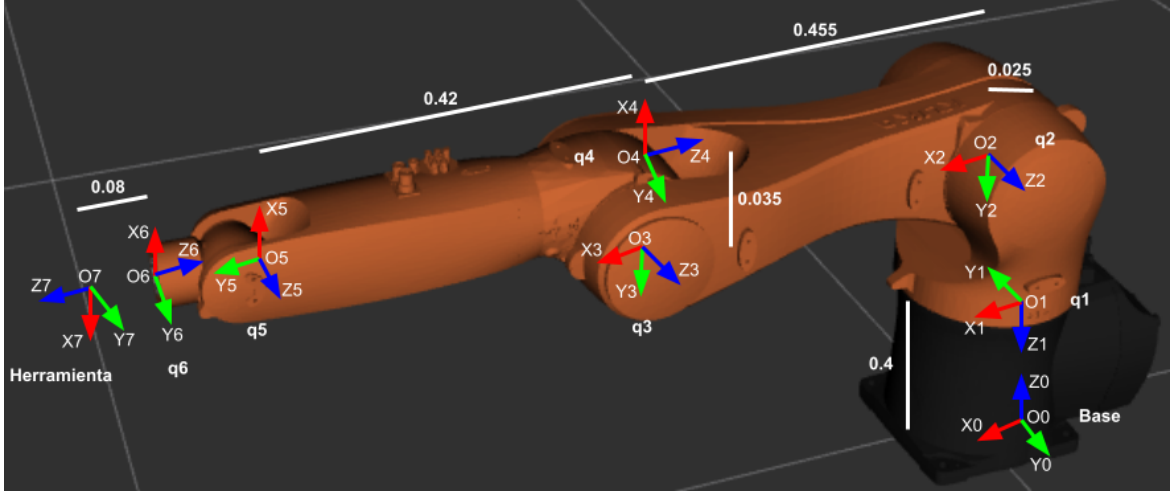


Figura 2. Asignación de marcos DH y dimensiones geométricas del KUKA KR 6 R900 sixx.

$$A_{i-1}^i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i). \quad (1)$$

Su forma matricial es:

$$A_{i-1}^i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

donde $c_{\theta_i} = \cos \theta_i$ y $s_{\theta_i} = \sin \theta_i$.

Se insertan en esta matriz general los valores obtenidos en la tabla de parámetros de Denavit–Hartenberg mostrada en la siguiente sección.

III. PARÁMETROS DENAVIT–HARTENBERG

En la siguiente tabla se muestran los parámetros DH siguiendo las transformaciones entre marcos de referencia de la figura 2.

Cuadro II
PARÁMETROS DENAVIT–HARTENBERG

T	θ	d	a	α
0-1	0	0.4	0	π
1-2	θ_1	0	0.025	$\pi/2$
2-3	θ_2	0	0.455	0
3-4	$\theta_3 - \pi/2$	0	0.035	$\pi/2$
4-5	θ_4	-0.42	0	$-\pi/2$
5-6	θ_5	0	0	$\pi/2$
6-7	$\theta_6 + \pi$	-0.08	0	π

III-A. Verificación geométrica de los parámetros

El cuadro muestra los puntos más importantes a considerar para aclarar la asignación de marcos de referencia mostrada anteriormente.

Cuadro III

JUSTIFICACIÓN DE LAS DISTANCIAS UTILIZADAS EN LOS PARÁMETROS DENAVIT–HARTENBERG

T	Justificación
0-1	Se considera la altura desde la base hasta la primera articulación.
1-2	El desplazamiento en X corresponde a la distancia horizontal entre la primera y segunda articulación.
2-3	No existe rotación entre las articulaciones 2 y 3, solo la distancia del eslabón.
3-4	Las articulaciones 3 y 4 se encuentran en el mismo punto horizontal, con una diferencia de altura que se plasma en el eje X debido a que este apunta hacia arriba tras la rotación de -90° en Z.
4-5	La distancia del eslabón se traduce a un desplazamiento negativo considerando la dirección del eje Z.
5-6	El nuevo eje Z (para la articulación 6) debe apuntar hacia adentro debido a que la herramienta apunta naturalmente hacia afuera. Por ende un giro de 90° .
6-7	El desplazamiento es negativo debido a la dirección de Z.

IV. CINEMÁTICA DIRECTA

La cinemática directa se obtiene mediante:

$$T_0^7 = A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6 A_6^7. \quad (3)$$

La transformación final tiene la forma:

$$T_0^7 = \begin{bmatrix} R_0^7 & p_0^7 \\ 0 & 1 \end{bmatrix}, \quad (4)$$

donde:

$$p_0^7 = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad R_0^7 \in SO(3). \quad (5)$$

IV-A. Matrices homogéneas individuales

Se define $c_i = \cos q_i$ y $s_i = \sin q_i$, donde q_i es el ángulo de la articulación i ; los offsets se incluyen explícitamente en la tabla DH. Sustituyendo los parámetros DH en la matriz general, se obtienen las siguientes transformaciones locales:

$$A_0^1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0.400 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (6)$$

$$A_1^2 = \begin{bmatrix} c_1 & 0 & s_1 & 0.025c_1 \\ s_1 & 0 & -c_1 & 0.025s_1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (7)$$

$$A_2^3 = \begin{bmatrix} c_2 & -s_2 & 0 & 0.455c_2 \\ s_2 & c_2 & 0 & 0.455s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (8)$$

$$A_3^4 = \begin{bmatrix} s_3 & 0 & -c_3 & 0.035s_3 \\ -c_3 & 0 & -s_3 & -0.035c_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (9)$$

$$A_4^5 = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & -0.420 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (10)$$

$$A_5^6 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (11)$$

$$A_6^7 = \begin{bmatrix} -c_6 & -s_6 & 0 & 0 \\ -s_6 & c_6 & 0 & 0 \\ 0 & 0 & -1 & -0.080 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (12)$$

IV-B. Matriz de transformación total

La transformación total se presenta desarrollada en la ecuación (13), a dos columnas. Se usan $c_{23} = \cos(q_2 + q_3)$ y $s_{23} = \sin(q_2 + q_3)$.

IV-C. Posición y orientación del efector

La posición del efector final corresponde a los primeros tres elementos de la última columna de la matriz de transformación total. Es decir, las posiciones para x, y, z se obtienen mediante las siguientes ecuaciones:

$$p_x(q) = -0.080s_1s_4s_5 - 0.080s_5s_{23}c_1c_4 + 0.035s_{23}c_1 \\ + 0.455c_1c_2 + 0.080c_1c_5c_{23} + 0.420c_1c_{23} + 0.025c_1 \quad (14)$$

$$p_y(q) = 0.080s_1s_5s_{23}c_4 - 0.035s_1s_{23} - 0.455s_1c_2 \\ - 0.080s_1c_5c_{23} - 0.420s_1c_{23} - 0.025s_1 - 0.080s_4s_5c_1 \quad (15)$$

$$p_z(q) = -0.455s_2 - 0.080s_5c_4c_{23} - 0.080s_{23}c_5 \\ - 0.420s_{23} + 0.035c_{23} + 0.400 \quad (16)$$

El producto de las siete matrices anteriores determina $T_0^7(q)$. Para expresar la posición se definen $c_{23} = \cos(q_2 + q_3)$ y $s_{23} = \sin(q_2 + q_3)$, junto con

$$U = 0.035 - 0.080c_4s_5, \quad (17)$$

$$V = 0.420 + 0.080c_5, \quad (18)$$

$$W = 0.080s_4s_5, \quad (19)$$

$$H = 0.025 + 0.455c_2 + Us_{23} + Vc_{23}. \quad (20)$$

Entonces,

$$p(q) = \begin{bmatrix} c_1H - s_1W \\ -s_1H - c_1W \\ 0.400 - 0.455s_2 + Us_{23} - Vc_{23} \end{bmatrix}. \quad (21)$$

La orientación se obtiene sin introducir nuevos cambios de marco:

$$R_0^7(q) = R_0^1R_1^2R_2^3R_3^4R_4^5R_5^6R_6^7, \quad (22)$$

donde R_a^b es el bloque de rotación de A_a^b . El nodo FK convierte esta matriz a un cuaternión unitario en orden (x, y, z, w) . Los cuaterniones de signo opuesto representan la misma orientación.

IV-D. Comprobaciones matemáticas

Para $q_0 = (0, -\pi/2, \pi/2, 0, 0, 0)^T$ se obtiene

$${}^B T_E(q_0) = \begin{bmatrix} 0 & 0 & 1 & 0.525 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0.890 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (23)$$

Así, $p = (0.525, 0, 0.890)^T$ m y el cuaternión es $(0, \sqrt{2}/2, 0, \sqrt{2}/2)$. Se verifica $R^T R = I$, $\det R = 1$ y la última fila $[0 \ 0 \ 0 \ 1]$, salvo residuos de punto flotante en la evaluación numérica.

IV-E. Comprobación numérica adicional de la cadena

Se evaluaron fuera de ROS las funciones matemáticas del commit indicado en la sección de reproducibilidad, usando exactamente los tres vectores $q^{(1)}, q^{(2)}, q^{(3)}$ transcritos en la validación FK. No se ejecutaron nodos ni se consultó TF. La expresión cerrada de la ecuación (13) y la cadena de siete matrices del código difieren como máximo en 2.23×10^{-16} por entrada. La última fila se mantiene igual a $[0 \ 0 \ 0 \ 1]$. Para

Cuadro IV
COMPROBACIÓN ALGEBRAICA FUERA DE ROS; NO ES ERROR RESPECTO DE TF.

Caso	$\ R^T R - I\ _F$	$ \det R - 1 $
1	3.16×10^{-16}	1.11×10^{-16}
2	5.78×10^{-16}	0
3	2.36×10^{-17}	1.11×10^{-16}

interpretar una orientación sin ambigüedad puede usarse la convención $R = R_z(\psi)R_y(\vartheta)R_x(\phi)$:

$$\vartheta = \text{atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2}), \quad (24)$$

$$\phi = \text{atan2}(r_{32}, r_{33}), \quad (25)$$

$$\psi = \text{atan2}(r_{21}, r_{11}). \quad (26)$$

$$\mathbf{T}_0^7 = \begin{bmatrix} -[(s_1 s_4 + s_{23} c_1 c_4) c_5 + s_5 c_1 c_{23}] c_6 & -[(s_1 s_4 + s_{23} c_1 c_4) c_5 + s_5 c_1 c_{23}] s_6 & -(s_1 s_4 + s_{23} c_1 c_4) s_5 & -0.080 s_1 s_4 s_5 - 0.080 s_5 s_{23} c_1 c_4 \\ & + (s_1 c_4 - s_4 s_{23} c_1) s_6 & + c_1 c_5 c_{23} & + 0.035 s_{23} c_1 + 0.455 c_1 c_2 \\ & & & + 0.080 c_1 c_5 c_{23} + 0.420 c_1 c_{23} \\ & & & + 0.025 c_1 \\ -[(-s_1 s_{23} c_4 + s_4 c_1) c_5 - s_1 s_5 c_{23}] c_6 & -[(-s_1 s_{23} c_4 + s_4 c_1) c_5 - s_1 s_5 c_{23}] s_6 & -(-s_1 s_{23} c_4 + s_4 c_1) s_5 & 0.080 s_1 s_5 s_{23} c_4 - 0.035 s_1 s_{23} \\ & + (s_1 s_4 s_{23} + c_1 c_4) s_6 & -s_1 c_5 c_{23} & -0.455 s_1 c_2 - 0.080 s_1 c_5 c_{23} \\ & & & -0.420 s_1 c_{23} - 0.025 s_1 \\ & & & -0.080 s_4 s_5 c_1 \\ -(-s_5 s_{23} + c_4 c_5 c_{23}) c_6 & -(-s_5 s_{23} + c_4 c_5 c_{23}) s_6 & -s_5 c_4 c_{23} & -0.455 s_2 - 0.080 s_5 c_4 c_{23} \\ & + s_4 s_6 c_{23} & -s_{23} c_5 & -0.080 s_{23} c_5 - 0.420 s_{23} \\ & & & + 0.035 c_{23} + 0.400 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (13)$$

Estas expresiones son válidas fuera del bloqueo de Euler ($\cos \vartheta = 0$). El programa conserva el cuaternión normalizado, que evita esa singularidad de representación. Esto no elimina las singularidades cinemáticas del robot.

V. JACOBIANO DEL MANIPULADOR

V-A. Jacobiano utilizado en el proyecto

La transformación anterior al movimiento de la articulación i , para $i = 1, \dots, 6$, es

$$G_i = A_0^1 \prod_{j=1}^{i-1} A_j^{j+1}, \quad G_1 = A_0^1. \quad (27)$$

El producto se toma en orden creciente y es la identidad para $i = 1$. Las tres primeras entradas de las columnas cuarta y tercera de G_i proporcionan el origen o_i y el eje unitario z_i , ambos expresados en la base. Cada columna del Jacobiano posicional es

$$j_i = z_i \times (p - o_i), \quad J_v = [j_1 \dots j_6]. \quad (28)$$

Este es el procedimiento de la función `jacobian_geometrico`; la IK utiliza sus primeras tres filas. Para expresar el resultado analíticamente se definen

$$H_2 = -0.455 s_2 + U c_{23} - V s_{23}, \quad (29)$$

$$H_3 = U c_{23} - V s_{23}, \quad (30)$$

$$H_4 = 0.080 s_4 s_5 s_{23}, \quad (31)$$

$$H_5 = -0.080 (c_4 c_5 s_{23} + s_5 c_{23}), \quad (32)$$

$$W_4 = 0.080 c_4 s_5, \quad W_5 = 0.080 s_4 c_5, \quad (33)$$

$$Z_2 = -0.455 c_2 - U s_{23} - V c_{23}, \quad (34)$$

$$Z_3 = -U s_{23} - V c_{23}, \quad (35)$$

$$Z_4 = 0.080 s_4 s_5 c_{23}, \quad (36)$$

$$Z_5 = -0.080 c_4 c_5 c_{23} + 0.080 s_5 s_{23}. \quad (37)$$

Las columnas son

$$j_1 = \begin{bmatrix} -s_1 H - c_1 W \\ -c_1 H + s_1 W \\ 0 \end{bmatrix}, \quad j_6 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}. \quad (38)$$

$$j_i = \begin{bmatrix} c_1 H_i \\ -s_1 H_i \\ Z_i \end{bmatrix} \quad (i = 2, 3). \quad (39)$$

$$j_i = \begin{bmatrix} c_1 H_i - s_1 W_i \\ -s_1 H_i - c_1 W_i \\ Z_i \end{bmatrix} \quad (i = 4, 5). \quad (40)$$

V-B. Verificación del Jacobiano por diferencias finitas

Se comprobó cada columna con

$$J_{FD}(:, i) = \frac{p(q + h e_i) - p(q - h e_i)}{2h}, \quad h = 10^{-6} \text{ rad}, \quad (41)$$

donde e_i es el vector canónico articular. En los tres vectores de prueba, el máximo error absoluto por entrada entre J_{FD} y el Jacobiano geométrico fue 7.27×10^{-11} , 7.99×10^{-11} y 1.09×10^{-10} m/rad. Esta prueba verifica la derivada del modelo implementado; no verifica por sí sola la fidelidad del URDF ni la ejecución en ROS.

V-C. Consideración de singularidades

Se utiliza un limitador de paso para recortar la magnitud del vector de movimiento, manteniendo intacta la dirección en la que el robot debe moverse. Cuando el robot se acerca a una configuración singular (como alinear las articulaciones de la muñeca o intentar estirarse más allá de su límite físico), el determinante del Jacobiano se acerca a cero. El código fuerza al robot a dar un micro-paso de solo 0.1 radianes en esa dirección. Esto le da tiempo al bucle de volver a recalcular un nuevo Jacobiano seguro en la siguiente iteración antes de que la simulación colapse.

```
norm_dq = np.linalg.norm(dq)
if norm_dq > max_step:
    dq = dq * (max_step / norm_dq)
```

Figura 3. Limitador de paso

VI. CINEMÁTICA INVERSA

VI-A. Definición del problema

La cinemática inversa implementada tiene como objetivo alcanzar una **posición cartesiana** deseada del efector final:

$$p_d = \begin{bmatrix} x_d \\ y_d \\ z_d \end{bmatrix}. \quad (42)$$

El algoritmo debe encontrar una configuración articular:

$$q^* = [q_1 \quad q_2 \quad \dots \quad q_6]^T \quad (43)$$

tal que:

$$p(q^*) \approx p_d. \quad (44)$$

VI-B. Objetivo recibido mediante ROS 2

Se utilizarán los siguientes puntos deseados como pruebas experimentales:

```
ros2 topic pub /target geometry_msgs/msg/Point \
"{x:_0.6,_y:_0.3,_z:_0.7}" --once
```

```
ros2 topic pub /target geometry_msgs/msg/Point \
"{x:_0.5,_y:_0.2,_z:_0.8}" --once
```

```
ros2 topic pub /target geometry_msgs/msg/Point \
"{x:_0.7,_y:_0.0,_z:_0.5}" --once
```

En la siguiente imagen se comprueba que el nodo IK recibe el objetivo con la posición en x, y, z:

```
class IKNode(Node):
    def __init__(self):
        super().__init__("ik iterative node")
        self.q = [0.0, -np.pi/2, np.pi/2, 0.0, 0.0, 0.0]
        self.have_solution = True

        self.publisher = self.create_publisher(JointState, "/joint_states", 10)

        self.subscription = self.create_subscription(
            Point, "/target", self.target_callback, 10
        )

        self.timer = self.create_timer(0.1, self.publish_solution)
        self.get_logger().info("Nodo IK activo. Esperando /target (Point)...")

    def target_callback(self, msg):
        p_des = np.array([msg.x, msg.y, msg.z], dtype=float)

        q_sol, success, iterations, error = ik_position_dls(p_des, self.q)

        if success:
            self.q = q_sol.tolist()
            self.have_solution = True
            self.get_logger().info(
                f"Convergencia en {iterations} iteraciones. "
                f"Ángulos: {[round(ang, 4) for ang in self.q]}"
            )
        else:
            self.get_logger().warning(
                f"Fallo de convergencia tras {iterations} iteraciones. "
                f"Error: {error:.6f} m. Reseteando semilla a 'home'."
            )
            self.q = [0.0, -np.pi/2, np.pi/2, 0.0, 0.0, 0.0]
```

Figura 4. Recepción del objetivo.

VI-C. Método numérico iterativo

A partir de la cinemática directa, se define el error de posición en la iteración k :

$$e_k = p_d - p(q_k). \quad (45)$$

El Jacobiano posicional se evalúa en la configuración actual:

$$J_k = J_v(q_k). \quad (46)$$

Antes de limitar el paso y aplicar los límites articulares, la actualización basada en la pseudoinversa es:

$$q_{k+1} = q_k + \alpha J_k^+ e_k, \quad (47)$$

donde α es el factor de actualización y J_k^+ es la pseudoinversa del Jacobiano.

Cuando el Jacobiano tiene rango de fila completo, puede expresarse como:

$$J^+ = J^T (J J^T)^{-1}. \quad (48)$$

La captura siguiente muestra el método implementado: se calcula explícitamente $J_v^T (J_v J_v^T)^{-1}$. Esta expresión representa

la pseudoinversa cuando J_v tiene rango de fila completo. No se incorpora amortiguamiento. Después se limita la norma del incremento, se aplica la ganancia y se recorta cada ángulo a sus límites.

```
for k in range(max_iterations):
    T_curr = fk(q)
    p_curr = T_curr[:3, 3]

    e_pos = p_des - p_curr

    if np.linalg.norm(e_pos) < tol_pos:
        return q, True, k, np.linalg.norm(e_pos)

    # Extraer posición de Jacobiano geométrico completo
    J = jacobian_geometrico(q)
    J_v = J[:3, :3]

    M = J_v @ J_v.T
    J_plus = J_v.T @ np.linalg.inv(M)
    dq = J_plus @ e_pos

    norm_dq = np.linalg.norm(dq)
    if norm_dq > max_step:
        dq = dq * (max_step / norm_dq)

    q = q + gain * dq
    q = aplicar_lmites(q)
```

Figura 5. Bloque iterativo.

VI-D. Configuración inicial

La configuración articular inicial es:

$$q_0 = [0 \quad -\pi/2 \quad \pi/2 \quad 0 \quad 0 \quad 0]^T \quad (49)$$

Esta es la configuración inicial elegida en el nodo y respeta sus límites. Tras converger, el programa reutiliza la última solución como semilla del siguiente objetivo. Por ello se debe registrar la semilla de cada ensayo; iniciar siempre desde q_0 exige reiniciar el nodo.

VI-E. Articulaciones utilizadas

Se utilizan las seis columnas del Jacobiano. La sexta es nula porque q_6 modifica la orientación sin desplazar el origen final; su valor inicial se conserva salvo residuos numéricos. Las demás articulaciones pueden variar y no se impone orientación objetivo.

VI-F. Límites articulares

Los límites proceden del modelo y de la ficha del robot [2], [1]. El código convierte grados a radianes con `numpy.deg2rad` y aplica `numpy.clip`; la tabla está redondeada.

Cuadro V
LÍMITES ARTICULARES EN RADIANES

Articulación	Mínimo [rad]	Máximo [rad]
q_1	-2.967060	2.967060
q_2	-3.316126	0.785398
q_3	-2.094395	2.722714
q_4	-3.228859	3.228859
q_5	-2.094395	2.094395
q_6	-6.108652	6.108652

VI-G. Criterio de convergencia

El error euclidiano de posición se define como:

$$e_p = \|p_d - p(q_k)\|_2. \quad (50)$$

El algoritmo se considera convergente cuando:

$$e_p < \varepsilon \quad (51)$$

antes de alcanzar el número máximo de iteraciones.

Se seleccionaron los siguientes valores para el criterio de convergencia:

$$\varepsilon = 10^{-3} \text{ m} \quad (52)$$

$$\alpha = 0.5 \quad (53)$$

$$\max_iteraciones = 200 \quad (54)$$

Antes de aplicar la ganancia, el código limita $\|\Delta q\|_2$ a 0.1 rad. Con $\alpha = 0.5$, el paso aplicado antes del recorte articular es como máximo 0.05 rad. La solución se acepta cuando el error euclidiano cae por debajo de la tolerancia.

VI-H. Publicación de la solución articular

Una vez obtenida la solución q^* , el nodo de cinemática inversa publica los valores articulares en:

/joint_states

mediante mensajes:

sensor_msgs/msg/JointState.

Esto permite que robot_state_publisher actualice la configuración del robot y que el resultado sea visible en RViz2.

En la siguiente imagen se muestra el bloque donde se publica el mensaje. En la función publish_solution se observa que el mensaje contiene los ángulos para las 6 articulaciones:

```
def target_callback(self, msg):
    p_des = np.array([msg.x, msg.y, msg.z], dtype=float)
    q_sol, success, iterations, error = ik_position(p_des, self.q)

    if success:
        self.q = q_sol.tolist()
        self.have_solution = True
        self.get_logger().info(
            f"Convergencia en {iterations} iteraciones. "
            f"Ángulos: {[round(ang, 4) for ang in self.q]}"
        )
    else:
        self.get_logger().warning(
            f"Fallo de convergencia tras {iterations} iteraciones. "
            f"Error={error:.6f} m. Reseteando semilla a 'home'."
        )
        self.q = [0.0, -np.pi/2, np.pi/2, 0.0, 0.0, 0.0]

def publish_solution(self):
    if not self.have_solution:
        return
    msg = JointState()
    msg.header.stamp = self.get_clock().now().to_msg()
    msg.name = [f"joint_{i}" for i in range(1, 7)]
    msg.position = self.q
    self.publisher.publish(msg)
```

Figura 6. Publicación de la solución en /joint_states.

VI-I. Verificación mediante cinemática directa

La solución obtenida por IK debe verificarse nuevamente mediante la cinemática directa:

$$p_f = p(q^*), \quad (55)$$

y comprobar:

$$\|p_d - p_f\|_2 < \varepsilon. \quad (56)$$

El nodo de FK calcula el error de la siguiente manera:

```
# Cálculo de error cuando es pertinente
if self.p_d is not None:
    p_f = np.array([x, y, z])
    error = np.linalg.norm(self.p_d - p_f)

self.get_logger().info(
    f" [Verificación IK] pd: ({self.p_d[0]:.4f}, {self.p_d[1]:.4f}, {self.p_d[2]:.4f}) | "
    f"pf: ({p_f[0]:.4f}, {p_f[1]:.4f}, {p_f[2]:.4f}) | Error: {error:.6f} m"
)
```

Figura 7. Cálculo de error de estimación de cinemática directa.

VII. IMPLEMENTACIÓN EN PYTHON Y ROS 2

VII-A. Herramientas

Se utilizan Ubuntu 24.04, ROS 2 Jazzy, Python 3, NumPy, RViz2, robot_state_publisher y joint_state_publisher_gui. Las funciones revisadas no requieren SymPy ni SciPy.

VII-B. Nodo de cinemática directa

El archivo fk_node.py, ejecutable fk, recibe /joint_states, evalúa matrices DH y calcula posición y cuaternión. Escucha también /target para informar objetivo, posición alcanzada y error. No usa TF como sustituto de la cinemática propia.

El nodo publicado toma las seis primeras posiciones en el orden recibido y comprueba que haya al menos seis valores. No realiza una asociación por msg.name; los mensajes deben conservar el orden joint_1,...,joint_6. Imprime la pose con tres decimales y no publica /fk_pose. Ordenar por nombre y registrar más decimales son mejoras pendientes.

VII-C. Nodo de cinemática inversa

El archivo ik_node.py, ejecutable ik, recibe geometry_msgs/msg/Point en /target y publica sensor_msgs/msg/JointState en /joint_states cada 0.1 s. Reporta convergencia, iteraciones y solución articular. FK reporta posición y error. Ante no convergencia, restablece la postura inicial y el temporizador continúa publicándola. No conserva la última solución ni registra un JSON con la semilla. Para comparar ensayos deben registrarse por separado los estados iniciales.

```
/target -> ik -> /joint_states
| -> robot_state_publisher -> RViz2
| -> fk -> verificacion numerica
```

VII-D. Interacción con Joint State Publisher GUI

Para FK manual se utiliza el GUI con IK detenido. Para IK se cierra el GUI antes de iniciar el nodo, evitando dos publicadores simultáneos de estados articulares.

VIII. WORKSPACE ROS 2

VIII-A. Estructura del proyecto

```
-- .gitignore
-- README.md
-- dependencias.repos
-- abrir.sh
-- entorno.sh
-- instalar.sh
-- recompilar.sh
-- verificar.sh
-- docs/
|   |-- informe_final.pdf
-- src/
|   |-- grupo03_kuka_kr6_bringup/
|   |   |-- launch/
|   |   |   |-- display.launch.py
|   |   |-- CMakeLists.txt
|   |   |-- package.xml
|   |-- grupo03_robot_kinematics/
|   |   |-- grupo03_robot_kinematics/
|   |   |   |-- __init__.py
|   |   |   |-- fk_node.py
|   |   |   |-- ik_node.py
|   |   |-- package.xml
|   |   |-- setup.cfg
|   |-- setup.py
```

La descripción externa se recupera en `src/kuka_robot_descriptions`. Las carpetas generadas no deben versionarse.

VIII-B. Compilación

Desde la raíz del workspace:

```
source /opt/ros/jazzy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

rosdep install --from-paths src --ignore-src -r -y

colcon build --symlink-install
source install/setup.bash
```

VIII-C. Ejecución del robot

```
ros2 launch grupo03_kuka_kr6_bringup \
  display.launch.py
```

VIII-D. Ejecución de los nodos propios

Terminal para cinemática directa:

```
ros2 run grupo03_robot_kinematics fk
```

Terminal para cinemática inversa:

```
ros2 run grupo03_robot_kinematics ik
```

En otra terminal se envía un objetivo:

```
ros2 topic pub /target geometry_msgs/msg/Point \
  "{x:_0.40,_y:_0.20,_z:_0.50}" --once
```

IX. VALIDACIÓN DE LA CINEMÁTICA DIRECTA

La referencia mostrada es la posición de `tool0` en RViz; para hacer la comparación plenamente reproducible debe registrarse también el marco fijo y el estado articular completo. La consola FK imprime tres decimales. Por ello, las diferencias siguientes corresponden a los valores visibles, no al error del cálculo antes de redondear.

IX-A. Primera prueba para cinemática directa

Para la primera prueba, se utiliza como referencia la siguiente configuración:

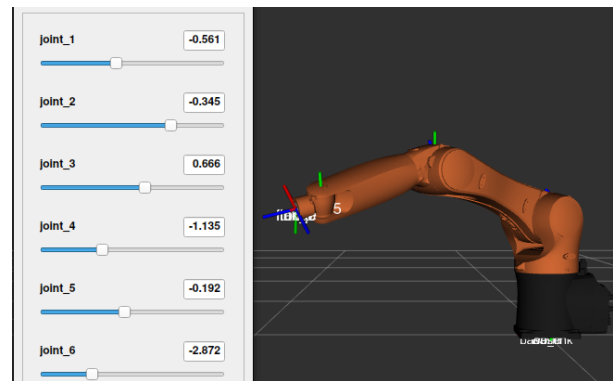


Figura 8. Primera configuración.

La cual produce la siguiente posición de la herramienta:

X	0.802553
Y	0.488323
Z	0.436379

Figura 9. Primera posición de herramienta.

Para dicha configuración, el nodo de cinemática directa produce la siguiente salida:

$$x = 0.803 \quad (57)$$

$$y = 0.488 \quad (58)$$

$$z = 0.436 \quad (59)$$

La diferencia entre las coordenadas impresas es aproximadamente:

$$e_{p,\text{impreso}} \approx 6.6916 \times 10^{-4} \text{ m} \quad (60)$$

IX-B. Segunda prueba para cinemática directa

Para la segunda prueba, se utiliza como referencia la siguiente configuración:

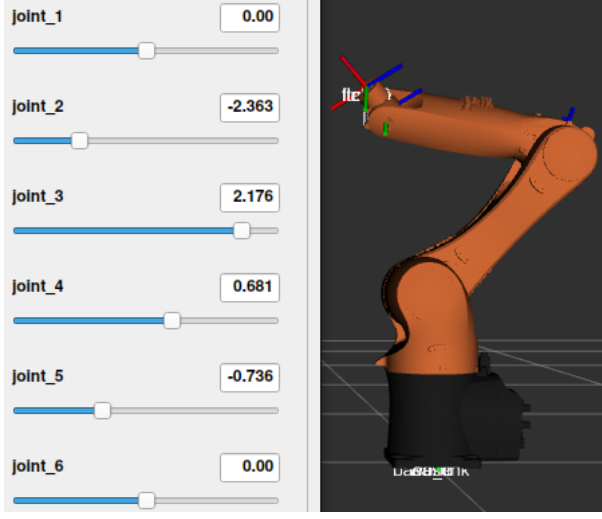


Figura 10. Segunda configuración.

La cual produce la siguiente posición de la herramienta:

X	0.157785
Y	0.0337959
Z	0.884047

Figura 11. Segunda posición de herramienta.

Para dicha configuración, el nodo de cinemática directa produce la siguiente salida:

$$x = 0.158 \quad (61)$$

$$y = 0.034 \quad (62)$$

$$z = 0.884 \quad (63)$$

La diferencia entre las coordenadas impresas es aproximadamente:

$$e_{p,\text{impreso}} \approx 3.0035 \times 10^{-4} \text{ m} \quad (64)$$

IX-C. Tercera prueba para cinemática directa

Para la tercera prueba, se utiliza como referencia la siguiente configuración:

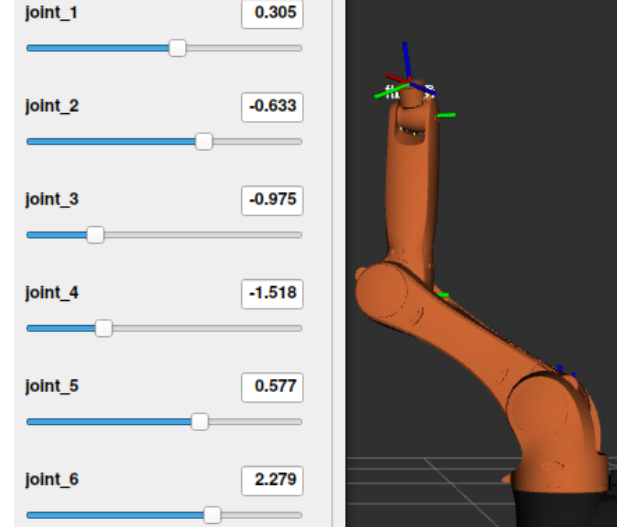


Figura 12. Tercera configuración.

La cual produce la siguiente posición de la herramienta:

X	0.338216
Y	-0.0607786
Z	1.15475

Figura 13. Tercera posición de herramienta.

Para dicha configuración, el nodo de cinemática directa produce la siguiente salida:

$$x = 0.338 \quad (65)$$

$$y = -0.061 \quad (66)$$

$$z = 1.155 \quad (67)$$

La diferencia entre las coordenadas impresas es aproximadamente:

$$e_{p,\text{impreso}} \approx 3.9771 \times 10^{-4} \text{ m} \quad (68)$$

Los vectores articulares visibles, en radianes, son:

$$\begin{aligned} q^{(1)} &= (0.272, -0.700, 1.109, \\ &\quad -1.483, 0.781, -2.542), \\ q^{(2)} &= (0.016, -2.496, 2.515, \\ &\quad 2.251, -1.030, -3.137), \\ q^{(3)} &= (0.272, -0.279, -1.339, \\ &\quad 0.157, -1.528, -5.118). \end{aligned}$$

Estos valores también están redondeados. Para completar la tabla con errores de orientación y posición exactos se requieren los mensajes originales o nuevas mediciones de TF y FK para el mismo estado.

Cuadro VI
COMPARACIÓN DE COORDENADAS VISIBLES EN LAS CAPTURAS FK.

Caso	p_{FK} impreso [m]	p_{RViz} impreso [m]	Error
1	(0.803, 0.488, 0.436)	(0.802553, 0.488323, 0.436379)	0.0006692
2	(0.158, 0.034, 0.884)	(0.157785, 0.0337959, 0.884047)	0.0003003
3	(0.338, -0.061, 1.155)	(0.338216, -0.0607786, 1.15475)	0.0003977

Cuadro VII
RESULTADOS DE LAS CAPTURAS ORIGINALES DE IK.

Caso	p_d [m]	p_f impreso [m]	Iteraciones	Error registrado [m]
1	(0.6, -0.3, 0.7)	(0.5998, -0.2995, 0.7003)	17	0.000626
2	(0.5, 0.2, 0.8)	(0.5000, 0.1994, 0.8002)	13	0.000606
3	(0.7, 0, 0.5)	(0.6995, 0, 0.5008)	17	0.000906

X. VALIDACIÓN DE LA CINEMÁTICA INVERSA

X-A. Primera prueba para cinemática inversa

Para el primer objetivo:

```
ros2 topic pub /target geometry_msgs/msg/Point \
  "{x:_0.6,_y:_0.3,_z:_0.7}" --once
```

Se obtiene la siguiente configuración mediante cinemática inversa:

```
[INFO] [1791429738.541137926] [ik_node]: Convergencia en 17 iteraciones.
Ángulos: [0.4631, -1.2112, 1.5359, 0.0003, -0.0106, -0.0]
```

Figura 14. Primera configuración IK.

La cual resulta en la siguiente posición y error mediante cinemática directa:

```
[INFO] [1791429742.913520104] [fk_node]: [Verificación IK] pd: (0.6000,
-0.3000, 0.7000) | pf: (0.5998, -0.2995, 0.7003) | Error: 0.000626 m
```

Figura 15. Error para el primer objetivo.

X-B. Segunda prueba para cinemática inversa

Para el segundo objetivo:

```
ros2 topic pub /target geometry_msgs/msg/Point \
  "{x:_0.5,_y:_0.2,_z:_0.8}" --once
```

Se obtiene la siguiente configuración mediante cinemática inversa:

```
[INFO] [1791430467.803637070] [ik_node]: Convergencia en 13 iteraciones.
Ángulos: [-0.3795, -1.5373, 1.7124, -0.0008, 0.024, -0.0]
```

Figura 16. Segunda configuración IK.

La cual resulta en la siguiente posición y error mediante cinemática directa:

```
[INFO] [1791430600.517363382] [fk_node]: [Verificación IK] pd: (0.5000,
0.2000, 0.8000) | pf: (0.5000, 0.1994, 0.8002) | Error: 0.000606 m
```

Figura 17. Error para el segundo objetivo.

X-C. Tercera prueba para cinemática inversa

Para el tercer objetivo:

```
ros2 topic pub /target geometry_msgs/msg/Point \
  "{x:_0.7,_y:_0.0,_z:_0.5}" --once
```

Se obtiene la siguiente configuración mediante cinemática inversa:

```
[INFO] [1791430713.794094015] [ik_node]: Convergencia en 17 iteraciones.
Ángulos: [-0.0, -0.9737, 1.6254, -0.0, 0.0009, 0.0]
```

Figura 18. Tercera configuración IK.

La cual resulta en la siguiente posición y error mediante cinemática directa:

```
[INFO] [1791430718.964856312] [fk_node]: [Verificación IK] pd: (0.7000,
0.0000, 0.5000) | pf: (0.6995, 0.0000, 0.5008) | Error: 0.000906 m
```

Figura 19. Error para el tercer objetivo.

X-D. Ensayos numéricos con semilla conocida

Para aislar el efecto de la inicialización, se ejecutó la función `ik_position` fuera de ROS, reiniciando $q_0 = (0, -\pi/2, \pi/2, 0, 0, 0)^T$ en cada objetivo. Se conservaron la ganancia, tolerancia, límite de paso y saturación del código. Los tres ensayos convergieron y respetaron los límites articulares. k cuenta actualizaciones del vector articular; $k = 0$ significaría aceptar la semilla sin actualizarla. Las soluciones numéricas adicionales son

$$q_{N1}^* = (0.463094110, -1.211159738, 1.535946027, 0.000301792, -0.010567134, 0),$$

$$q_{N2}^* = (-0.379538699, -1.537326039, 1.712380119, -0.000818015, 0.023952786, 0),$$

$$q_{N3}^* = (0, -0.973688193, 1.625352306, 0, 0.000877102, 0).$$

Se muestran como cero los residuos de orden 10^{-17} o menores. El segundo resultado coincide, a la precisión publicada, con el ejemplo del README actual (error 0.000606 m); difiere de la captura original de 19 iteraciones y 0.000839 m. Por ello se mantienen ambas evidencias identificadas y no se atribuye retrospectivamente una semilla a aquella captura.

Cuadro VIII
ENSAYOS ADICIONALES FUERA DE ROS

Caso	p_d [m]	$p(q^*)$ [m]	k / error [m]
N1	(0.6, -0.3, 0.7)	(0.599811186, -0.299490429, 0.700310895)	17 / 0.000626074
N2	(0.5, 0.2, 0.8)	(0.500013984, 0.199446229, 0.800246808)	13 / 0.000606442
N3	(0.7, 0, 0.5)	(0.699548356, 0, 0.500785795)	17 / 0.000906342

X-E. Protocolo para cerrar la validación en ROS

La comparación debe usar `base_link` como base y `tool0` como extremo, con el robot detenido y el mismo estado articular en FK y TF. Los cuaterniones de ambas fuentes deben normalizarse y expresarse en ese mismo par de marcos. El error de orientación se calcula como

$$e_R = 2 \arccos \left(\min \{1, |u_{FK}^T u_{TF}|\} \right), \quad (69)$$

en radianes. El valor absoluto elimina la ambigüedad de signo del cuaternión. No corresponde comparar directamente sus cuatro componentes sin considerar esa equivalencia.

Para cada prueba FK se guardarán q , posición y cuaternión calculados, posición y cuaternión de TF, sus marcas temporales, e_p y e_R . La posición y orientación de FK deben imprimirse con al menos nueve decimales antes de registrar los datos. Una captura con tres decimales no permite recuperar la precisión perdida. Para cada prueba IK se registrarán también q_0 , q^* , objetivo, número de actualizaciones, convergencia y error final. Reiniciar IK antes de cada objetivo fija la semilla inicial; si se usa continuidad, se guarda la solución anterior completa.

Comandos de apoyo, con el entorno cargado:

```
ros2 topic info /joint_states --verbose
ros2 topic echo /joint_states --once
ros2 run tf2_ros tf2_echo base_link tool0
ros2 bag record -o validacion_grupo3 \
  /joint_states /target /tf /tf_static
```

Se verifica un único publicador activo de estados articulares durante cada ensayo. La grabación debe iniciarse antes de las pruebas y finalizarse con Ctrl+C. El registro de tópicos conserva datos de ROS, pero los mensajes de consola con semillas e iteraciones deben guardarse adicionalmente. Este protocolo queda propuesto para la sesión experimental; no se afirma que esa grabación ya exista.

XI. REPOSITORIO GITHUB Y REPRODUCIBILIDAD

XI-A. Enlace y versión revisada

https://github.com/FernandoAramayo/kinematics_kuka_kr6_r900_sixx_ws

XI-B. Procedimiento de reproducción

Se requiere Ubuntu 24.04 con ROS 2 Jazzy instalado; el script no instala ROS desde cero.

```
git clone https://github.com/FernandoAramayo/
  kinematics_kuka_kr6_r900_sixx_ws
cd kinematics_kuka_kr6_r900_sixx_ws

chmod +x instalar.sh abrir.sh recompilar.sh
  verificar.sh
./instalar.sh
```

Compilación:

```
source /opt/ros/jazzy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
source install/setup.bash
```

O mediante el atajo `source entorno.sh`.

Para visualizar el robot:

```
ros2 launch grupo03_kuka_kr6_bringup display.launch.py
```

Para correr la cinemática, en terminales separadas desde la raíz del workspace:

```
source entorno.sh
ros2 run grupo03_robot_kinematics fk
```

```
source entorno.sh
ros2 run grupo03_robot_kinematics ik
```

Cerrar el GUI antes de iniciar IK. Cargar el entorno también en la terminal donde se publican los objetivos.

XI-C. Dependencias y estado de reproducción

El archivo `dependencias.repos` publicado apunta a la rama master de https://github.com/kroshu/kuka_robot_descriptions. No fija una versión inmutable. Debe registrarse el commit realmente usado en las pruebas y fijarlo en ese archivo.

XI-D. Reproducción de las pruebas numéricas

En un entorno ROS 2 compilado y cargado, el siguiente bloque repite N1–N3 sin iniciar nodos. Importa las funciones existentes en `ik_node.py`; requiere las bibliotecas ROS, aunque no se llama a `main()`:

```
source entorno.sh
python3 - <<'PY'
import numpy as np
from grupo03_robot_kinematics.ik_node import (
    ik_position, fk, Q_MIN, Q_MAX)
q0 = np.array([0., -np.pi/2, np.pi/2, 0., 0., 0.])
for p in ([.6, -.3, .7], [.5, .2, .8], [.7, 0., .5]):
    pd = np.array(p)
    q, ok, k, error = ik_position(pd, q0.copy())
    print('q0=', q0.tolist(), 'pd=', p)
    pf = fk(q)[:3,3]
    print('q=', q.tolist())
    print('pf=', pf.tolist())
    print('converge=', ok, 'k=', k, 'error=', error)
    print('limites=', bool(np.all(q >= Q_MIN)
                          and np.all(q <= Q_MAX)))
PY
```

El bloque anterior repite la parte numérica de IK con la semilla declarada. No inicia RViz ni verifica mensajes. Para repetir además la comparación independiente con XACRO, descargar `validar_informe.py` del material complementario y ejecutarlo desde la raíz del workspace:

```
DESC=src/kuka_robot_descriptions
MODEL=$DESC/kuka_agilus_support/urdf
python3 validar_informe.py . \
  "$MODEL/kr6_r900_sixx_macro.xacro" \
  > resultados_repeticion.json
```

Debe usarse el commit de la descripción indicado anteriormente. Esta ejecución solo requiere NumPy: extrae las funciones matemáticas del código mediante el árbol de sintaxis y no importa ni inicia nodos ROS.

XI-E. Archivos excluidos

No se versionan `build/`, `install/`, `log/`, `__pycache__`/ ni archivos generados. La entrega debe incluir fuentes y datos de prueba, no sólo un workspace comprimido.

XII. RESULTADOS Y DISCUSIÓN

Las capturas IK reportan inferiores a la tolerancia de 1 mm (cuadro VI). Estos resultados confirman que la cinemática inversa mediante el método iterativo funciona de manera óptima, convergiendo cuando el error es menor a la tolerancia establecida. Las tres pruebas realizadas para el proceso completo (cinemática inversa + validación por cinemática directa) demuestran que el proyecto es reproducible. El procedimiento para realizar pruebas como estas están listados de manera detallada en el repositorio de GitHub.

XIII. CONCLUSIONES

1. Se obtuvo una formulación DH coherente con el código, incluyendo las transformaciones fijas necesarias para interpretar posición y orientación.
2. Se logró implementar mediante código el desarrollo del jacobiano geométrico con resultados favorables.
3. Los tres ensayos adicionales desde q_0 convergieron en 17, 13 y 17 actualizaciones, con errores de 0.626, 0.606 y 0.906 mm. Son comprobaciones numéricas fuera de ROS; las capturas originales aún requieren registrar sus semillas.
4. Futuras mejoras para el proyecto incluyen métodos más avanzados para tratamiento de singularidades e implementación de DLS.

REFERENCIAS

- [1] KUKA, KR AGILUS: especificaciones técnicas, https://www.kuka.com/-/media/kuka-downloads/imported/6b77eeca542d3b736af377562ecaa/pb0001_kr_agilus_en.pdf.
- [2] kroshu, KUKA robot descriptions: KR6 R900 sixx, https://github.com/kroshu/kuka_robot_descriptions/blob/d2fa4377b77b7370cdcf6e52b430fc514a0eef19/kuka_agilus_support/urdf/kr6_r900_sixx_macro.xacro.
- [3] K. M. Lynch y F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge University Press, 2017.
- [4] Grupo 3, código del proyecto, https://github.com/FernandoAramayo/kinematics_kuka_kr6_r900_sixx_ws. Versión revisada indicada en la sección de reproducibilidad.

APÉNDICE A

ALCANCE DE LA ENTREGA Y COMPROBACIÓN EN ROS

La evidencia disponible comprende el código publicado, las capturas originales, las tablas numéricas y los archivos complementarios `validar_informe.py` y `resultados_informe.json` incluidos en el proyecto Overleaf. Las semillas de las capturas históricas no pueden reconstruirse a partir de sus imágenes.

Para completar la entrega deben publicarse el PDF, las fuentes y los scripts y datos de las comprobaciones adicionales. La validación sincronizada debe comparar FK y TF para el mismo estado y par de marcos. La repetición experimental y una instalación limpia requieren Ubuntu 24.04 y ROS 2 Jazzy.